KPR Institute of Engineering and Technology

Department of Artificial Intelligence and Data Science

U21ADP05 - Exploratory Data Analysis and Visualization

Assignment II - Case Study

Exploratory Data Analysis and Visualization of the Breast Cancer Wisconsin (Diagnostic) Dataset

with a Deep Learning (MLP) Classifier

Item	Details
Student Name	YOUR NAME
Roll No.	YOUR ROLL NO
Department	Artificial Intelligence and Data Science
Year / Semester	III Year / V Semester (Batch 2023-2027)
Course Coordinator	Mr. Rushikesh Kadam
Date of Submission	DD/MM/2026
GitHub Repository	https://github.com/<your-username>/breast-cancer-eda-mlp

Abstract

Early and accurate classification of breast masses as malignant or benign supports timely clinical decisions. This case study applies exploratory data analysis (EDA) and visualization to the Breast Cancer Wisconsin (Diagnostic) dataset, containing 569 patients and 30 numerical features computed from digitized fine-needle-aspirate images. The data were checked for missing values, duplicates and outliers, standardized, and split into train, validation and test sets. Nine visualizations covering all seven visualization stages revealed strong class separation and heavy multicollinearity among size-related features. A multilayer perceptron with hyperparameter tuning achieved 96.5% test accuracy and an AUC of 0.990. Size and concavity features proved most informative, and a logistic regression baseline performed comparably.

(Word count: 109)

Keywords: EDA, data visualization, breast cancer, multilayer perceptron, ROC-AUC

1. Introduction and Objective

Breast cancer is one of the most common cancers worldwide, and the outcome for a patient depends strongly on how early and how reliably a tumour is identified as malignant. Fine-needle aspiration (FNA) is a minimally invasive procedure in which cells are extracted from a breast mass and examined under a microscope. Digitizing the slide image and measuring the shape of each cell nucleus gives quantitative features that can be analysed statistically and used to train predictive models [1].

Real-world data of this type is rarely ready for modelling: features are on different scales, strongly correlated, skewed, and contain extreme values. Exploratory data analysis and visualization are therefore the first and most important step, because they reveal what the data can and cannot support before any model is trained.

1.1 Objectives

- Select a real-world dataset with at least 15 features and describe its source, size and fields.
- Perform EDA: handle missing values, duplicates and outliers; transform, scale and split the data.
- Create at least five meaningful visualizations covering the seven stages of visualization, each explained.
- Analyse univariate distributions (discrete and continuous), and bivariate / multivariate relationships.
- Build, tune and evaluate a deep learning model (MLP) and interpret loss, accuracy, confusion matrix, ROC and AUC.
- Draw practical conclusions and suggest future improvements.

2. Dataset Description

The Breast Cancer Wisconsin (Diagnostic) dataset (WDBC) was created by Wolberg, Street and Mangasarian at the University of Wisconsin and is hosted in the UCI Machine Learning Repository [1], [2]. It is also bundled with scikit-learn [3], which was used to load it. Features describe characteristics of cell nuclei in a digitized FNA image.

Table 1: Dataset overview

Property	Value
Source	UCI Machine Learning Repository / scikit-learn (load_breast_cancer)
Records (patients)	569
Input features	30
Target	diagnosis: 0 = Malignant, 1 = Benign
Class counts	Malignant = 212 (37.3%), Benign = 357 (62.7%)
Feature type	All continuous (real-valued)
Missing values	0
Duplicate rows	0

Ten base measurements are computed for every nucleus: radius, texture, perimeter, area, smoothness, compactness, concavity, concave points, symmetry and fractal dimension. For each, three versions are stored - the mean, the standard error (SE) and the 'worst' (mean of the three largest values) - giving 10 x 3 = 30 features.

Table 2: Basic statistics of selected features (full table in results/summary_statistics.csv)

Feature	Mean	Std	Min	Median	Max
mean radius	14.127	3.524	6.981	13.370	28.110
mean texture	19.290	4.301	9.710	18.840	39.280
mean area	654.889	351.914	143.500	551.100	2501.000
mean smoothness	0.096	0.014	0.053	0.096	0.163
mean concavity	0.089	0.080	0.000	0.062	0.427
worst area	880.583	569.357	185.200	686.500	4254.000

3. EDA and Preprocessing

3.1 Methods used

- **Loading and inspection:** Dataset loaded into a pandas DataFrame (569 x 31); data types, shape and summary statistics inspected.
- **Missing values and duplicates:** No missing values (0) and no duplicate rows (0) were found, so no imputation or removal was needed.
- **Outliers:** The 1.5 x IQR rule flagged 608 values across all features; the most affected were area error (65), radius error (38), perimeter error (38), worst area (35), smoothness

error (30). Because such values correspond to genuinely large tumours (clinically meaningful), they were not deleted. Instead, extreme values beyond 3 x IQR (fences computed on the training set only) were clipped - 82 values - to limit their influence on the neural network.

- **Encoding:** The target is already numerically encoded (0/1), and all 30 predictors are numerical, so no one-hot encoding was required.
- **Scaling:** Features were standardized to zero mean and unit variance with StandardScaler, fitted on the training set only to avoid data leakage.
- **Splitting:** A stratified 70/15/15 split gave 398 training, 85 validation and 86 test samples, preserving the class ratio in every subset.

3.2 Insights from EDA

- The classes are moderately imbalanced (about 63% benign, 37% malignant), so accuracy alone is not enough - recall, precision and AUC are also reported.
- Several features are strongly right-skewed (e.g., area error (5.45), concavity error (5.11), fractal dimension error (3.92)); Shapiro-Wilk tests rejected normality ($p < 0.05$) for 30 of 30 features.
- Welch t-tests found a significant difference between malignant and benign means ($p < 0.05$) for 26 of 30 features, confirming that most features carry diagnostic information.
- Size features are almost perfectly correlated - mean radius | mean perimeter ($r = 0.998$); worst radius | worst perimeter ($r = 0.994$); mean radius | mean area ($r = 0.987$) - indicating severe multicollinearity.

4. Data Visualization

The visualizations follow the seven stages of data visualization described by Fry [6]. Table 3 maps each stage to the work done in the project.

Table 3: Mapping of the seven visualization stages

Stage	Activity in this project	Output
1. Acquire	Loaded WDBC dataset from scikit-learn / UCI and cached as CSV	data/breast_cancer_wisconsin.csv
2. Parse	Inspected structure, types, missing values, duplicates, summary statistics	summary_statistics.csv; Tables 1-2
3. Filter	Removed duplicates (none), identified IQR outliers, kept clinically valid values	Section 3.1
4. Mine	Skewness, Shapiro-Wilk, t-tests, correlation, mutual information	Section 3.2; Figure 8
5. Represent	Selected chart types by data type (bar, histogram, box, heatmap, scatter, pair, Q-Q)	Figures 1-8
6. Refine	Titles, labels, colour-blind-friendly class colours, before/after scaling comparison	Figure 9

7. Interact	Interactive Plotly scatter (generated when plotly is installed)	results/interactive_scatter.html

Class distribution

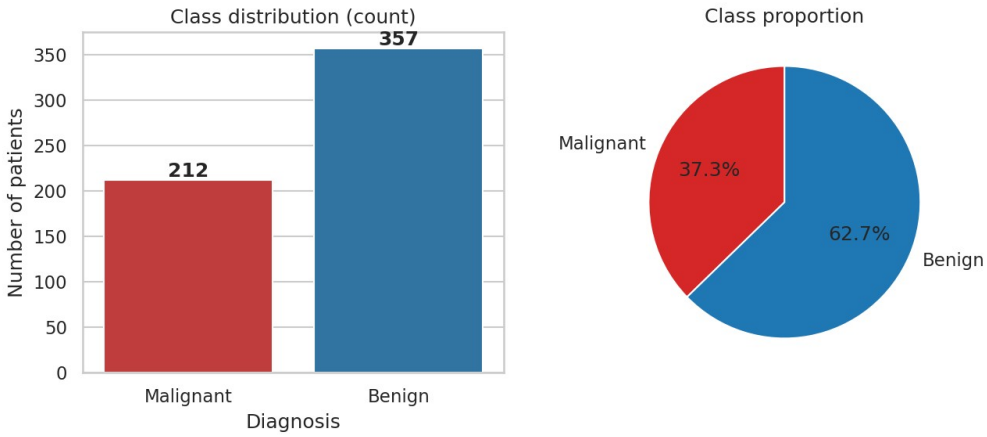


Figure 1: Class distribution

What it shows: A bar chart and pie chart of the number of malignant and benign patients (discrete target variable).

Why it was created: To check class balance before modelling, since imbalance affects the choice of evaluation metrics. The target follows a Bernoulli distribution for each patient and a Binomial distribution for counts.

Insights obtained: There are 357 benign (62.7%) and 212 malignant (37.3%) cases - moderately imbalanced but not severely, so stratified splitting is sufficient.

Histograms with fitted normal curve

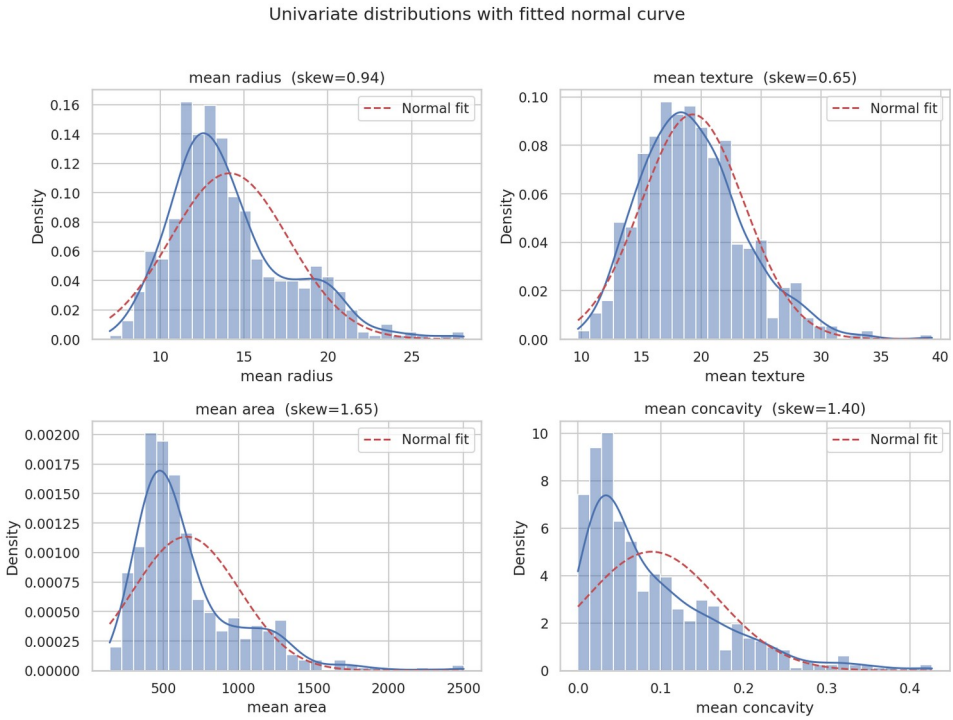


Figure 2: Histograms with fitted normal curve

What it shows: Density histograms with a KDE and a fitted normal curve for mean radius, mean texture, mean area and mean concavity.

Why it was created: To study each continuous variable's distribution (univariate analysis) and compare it with the normal distribution.

Insights obtained: Mean radius and mean texture are roughly bell-shaped (with mild right skew), whereas mean area and mean concavity are clearly right-skewed with long tails, so the normal curve fits them poorly. This supports scaling and outlier control before neural-network training.

Boxplots by diagnosis

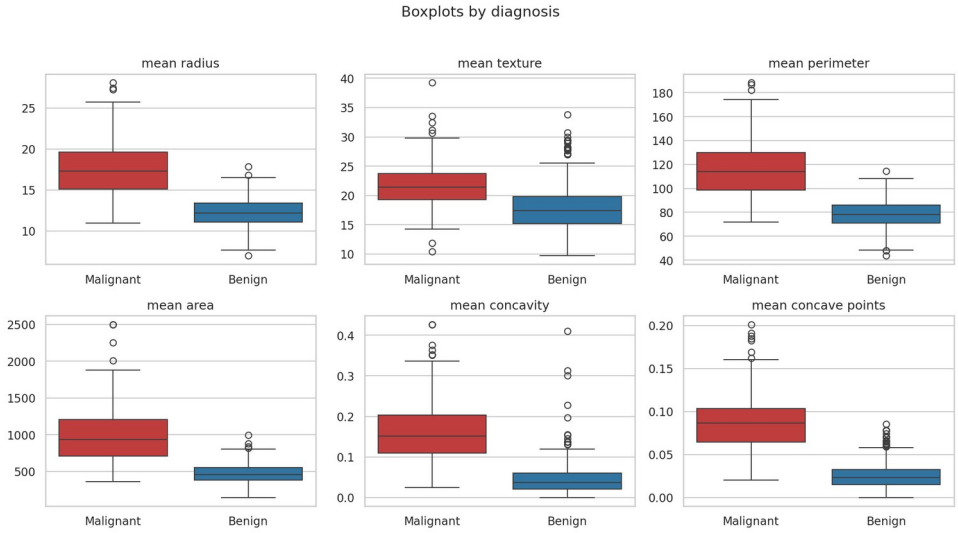


Figure 3: Boxplots by diagnosis

What it shows: Boxplots of six 'mean' features, split by malignant and benign.

Why it was created: To compare class medians and spreads and to spot outliers.

Insights obtained: Malignant tumours have clearly higher radius, perimeter, area, concavity and concave points; the boxes overlap little for concave points and perimeter, which makes these good discriminators. Texture overlaps more. Several points lie beyond the whiskers, i.e., statistical outliers.

Correlation heatmap

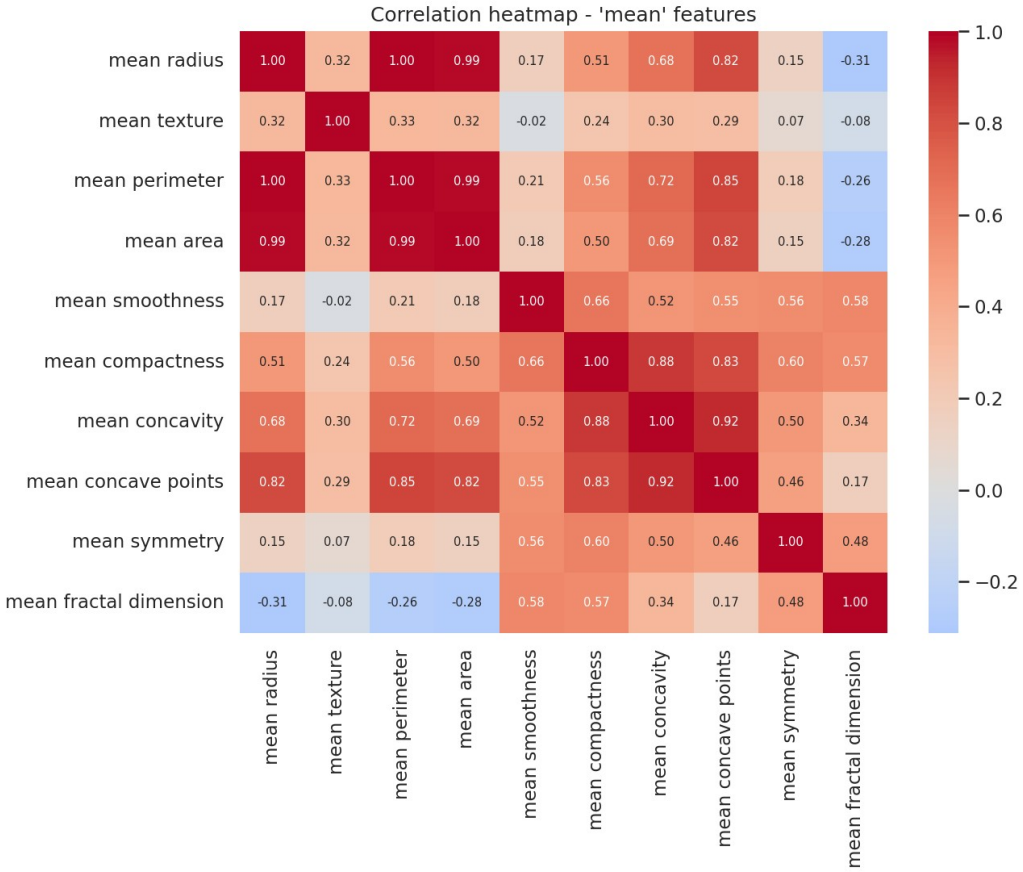


Figure 4: Correlation heatmap

What it shows: Pearson correlation coefficients between the ten 'mean' features.

Why it was created: To detect multicollinearity and relationships among variables (multivariate analysis).

Insights obtained: Radius, perimeter and area are almost perfectly correlated (r above 0.98) because they are geometrically related; concavity, concave points and compactness form a second correlated group. Smoothness and fractal dimension are weakly related to size. Redundant features justify regularization (dropout) in the model.

Scatterplot: radius vs concave points

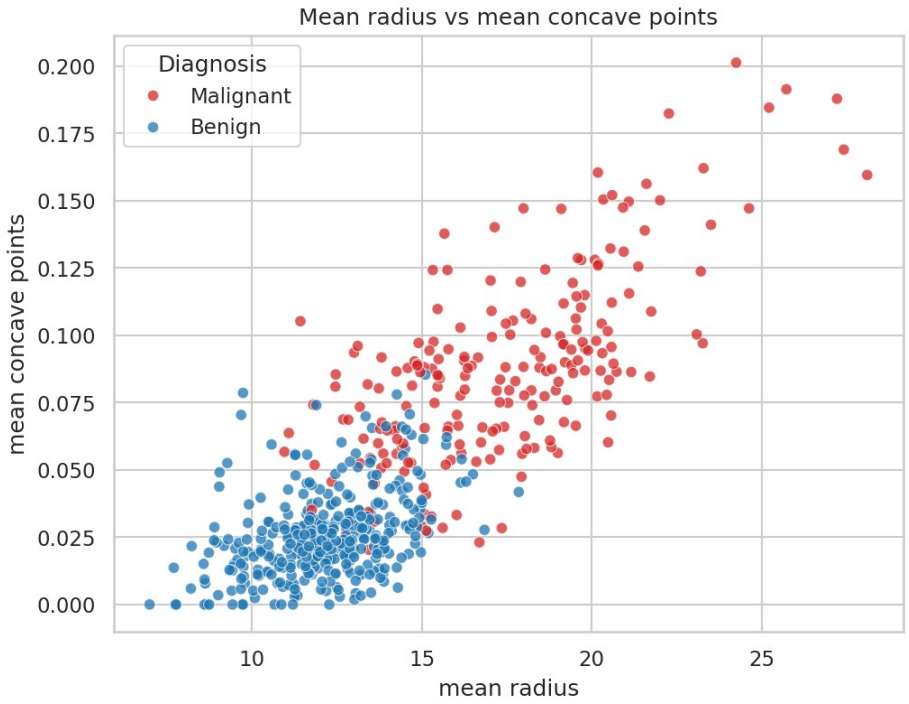


Figure 5: Scatterplot: radius vs concave points

What it shows: Each patient plotted by mean radius (x) and mean concave points (y), coloured by diagnosis.

Why it was created: To visualize bivariate class separation in two of the most discriminating features.

Insights obtained: The two classes form largely separate clusters: malignant cases sit at larger radius and higher concave-points values. Only a narrow overlapping zone exists, which explains why even simple models perform well on this dataset.

Pairplot of the five most informative features

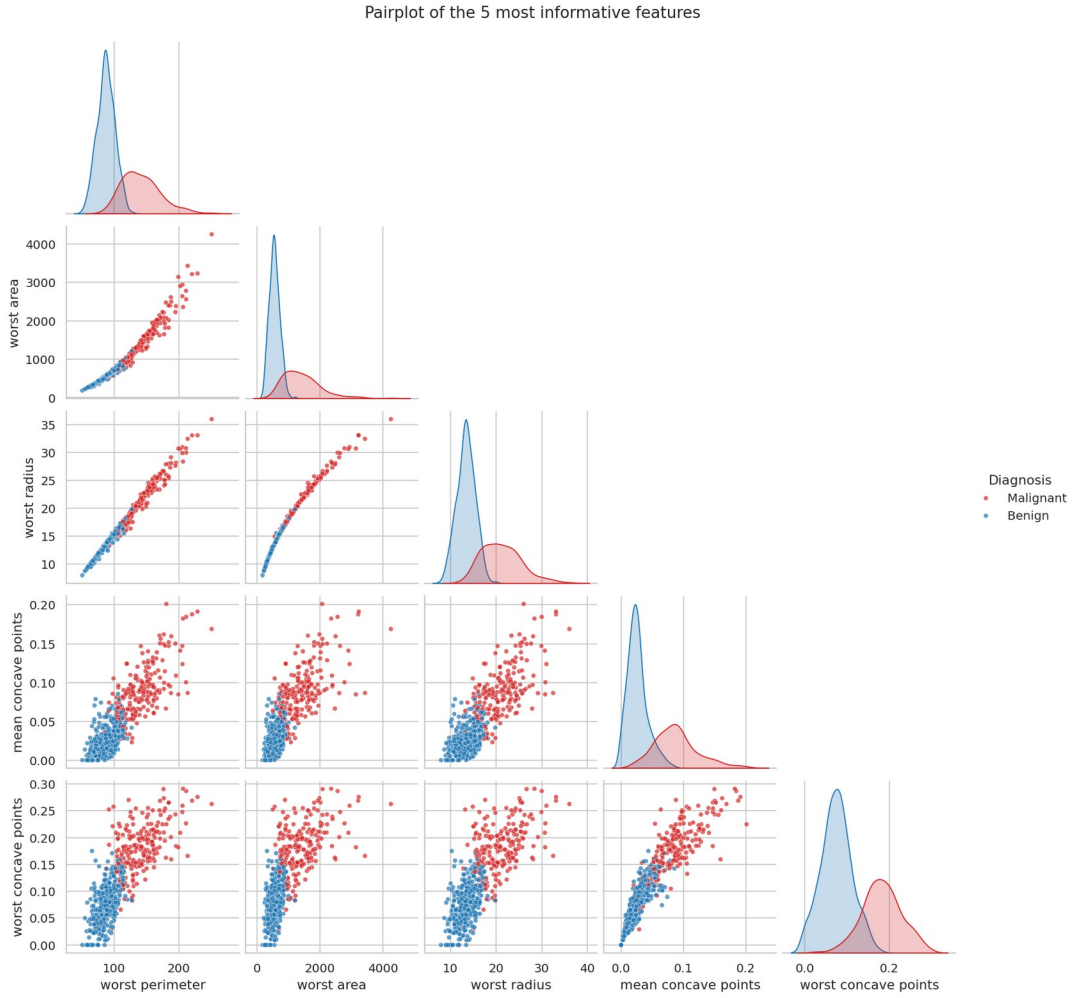


Figure 6: Pairplot of the five most informative features

What it shows: Pairwise scatterplots and diagonal histograms of the top five features ranked by mutual information.

Why it was created: To examine multivariate structure and class separation across several features at once.

Insights obtained: Worst perimeter, worst area and worst radius form near-linear relationships (redundant), and all five features separate the classes along the diagonal distributions.

Q-Q plots

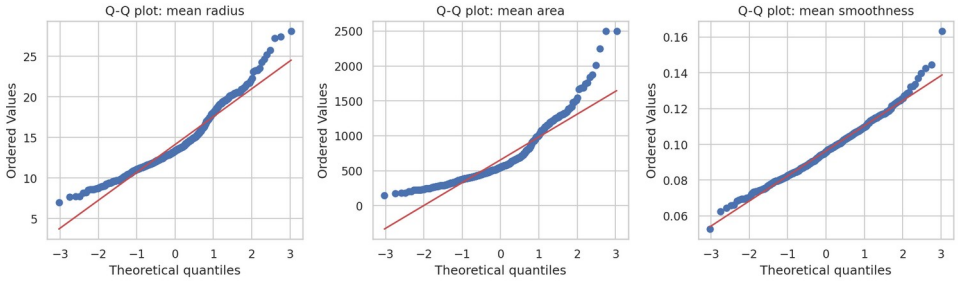


Figure 7: Q-Q plots

What it shows: Quantile-quantile plots of mean radius, mean area and mean smoothness against a normal distribution.

Why it was created: To assess normality of continuous variables more precisely than a histogram.

Insights obtained: Points far from the red line in the upper tail of mean area show a right-skewed (approximately log-normal) distribution, while mean smoothness is closer to normal. Features that deviate from normality benefit from standardization.

Mutual information with the diagnosis

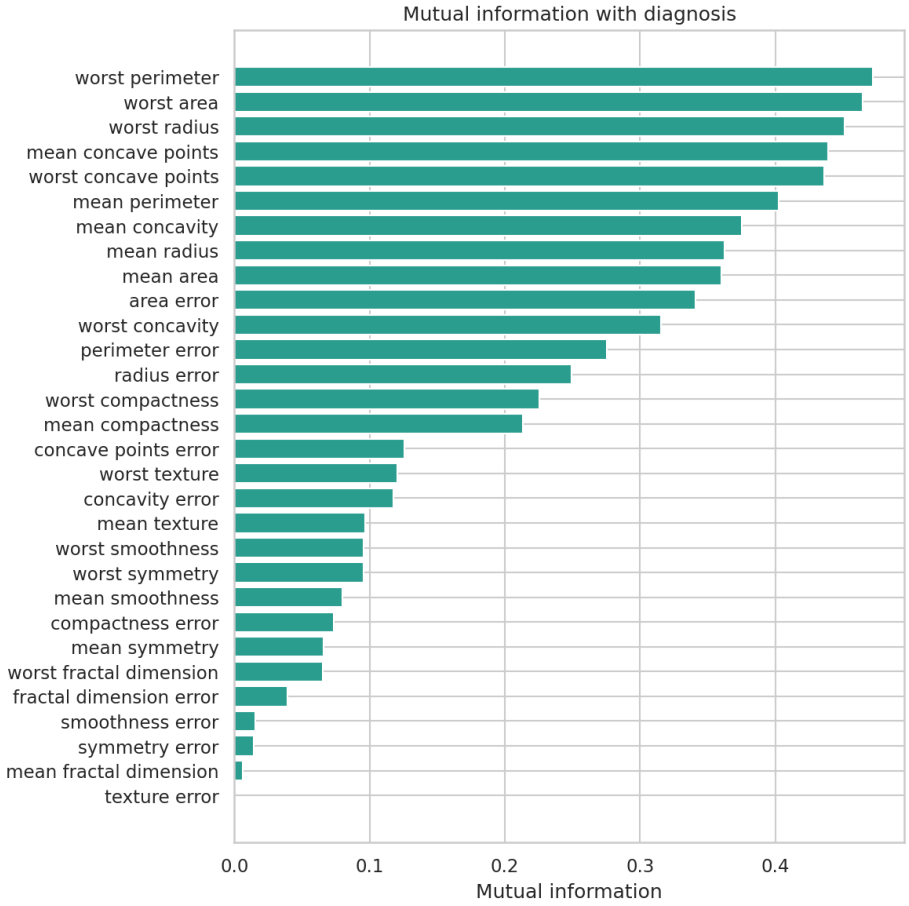


Figure 8: Mutual information with the diagnosis

What it shows: Horizontal bar chart of the mutual information between each feature and the diagnosis.

Why it was created: To rank features by how much information they carry about the target without assuming linearity (feature analysis).

Insights obtained: The top features are worst perimeter, worst area, worst radius, mean concave points, worst concave points. 'Worst' and concave-point features dominate, while several standard-error features and fractal-dimension features carry little information.

Effect of standardization

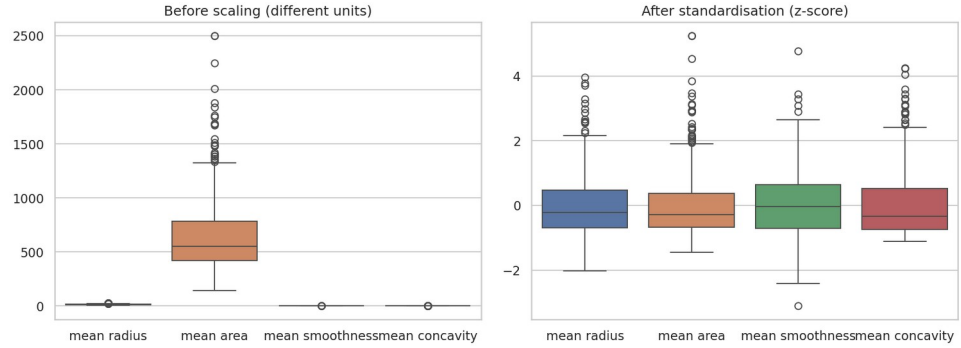


Figure 9: Effect of standardization

What it shows: Boxplots of four features before and after z-score standardization.

Why it was created: To refine the data representation and show why scaling is required (the 'Refine' stage).

Insights obtained: Before scaling, mean area (hundreds to thousands) dwarfs smoothness (about 0.1), which would dominate gradient updates. After standardization all features share a comparable scale.

4.1 Discrete and continuous probability distributions

- **Discrete:** The diagnosis of each patient is a Bernoulli trial (benign probability about 0.63); the number of benign cases in a sample follows a Binomial distribution.
- **Continuous:** Mean radius and mean texture are roughly normal (mild right skew); area, concavity and error features are right-skewed and closer to log-normal. Normal-curve overlays (Figure 2), Q-Q plots (Figure 7) and Shapiro-Wilk tests were used for identification.

5. Deep Learning Model

A Multilayer Perceptron (MLP) was chosen because the data is tabular with numerical features. Implementation: NumPy MLP (TensorFlow not installed).

5.1 Architecture

Table 4: MLP architecture

Layer	Type	Output units	Activation / Regularization
Input	Input	30	Standardized features
Hidden 1	Dense + Dropout	32	ReLU, dropout = 0.2
Hidden 2	Dense + Dropout	16	ReLU, dropout = 0.2
Output	Dense	1	Sigmoid (probability of Benign)

5.2 Training parameters

Table 5: Training parameters

Parameter	Value
Optimizer	Adam (learning rate = 0.01)
Loss	binary cross-entropy
Batch size	32
Maximum epochs	100
Early stopping	monitor val_loss, patience = 15, restore best weights
Epochs actually run / best epoch	30 / 15
Random seed	42

5.3 Hyperparameter tuning

A basic grid search over hidden-layer sizes, dropout rate and learning rate (12 configurations) was run. Each model was trained on the training set and compared on the validation set; the test set was not used for tuning. The configuration with the lowest validation loss was selected. Because the validation set is small (85 samples), differences of one or two samples in accuracy are not statistically meaningful.

Table 6: Top 6 hyperparameter configurations (sorted by validation loss)

Hidden units	Dropout	Learning rate	Best val. loss	Best val. acc.	Epochs
(32, 16)	0.2	0.01	0.0094	1.0	30
(32, 16)	0.3	0.01	0.0156	0.9882	30
(16, 8)	0.3	0.01	0.0234	1.0	39
(64, 32)	0.3	0.01	0.0303	0.9882	37
(64, 32)	0.3	0.001	0.0341	0.9765	78
(64, 32)	0.2	0.01	0.0342	0.9882	19

5.4 Feature analysis and importance

Feature importance was computed in two ways: (i) permutation importance of the trained MLP (the drop in validation accuracy when one feature is shuffled) and (ii) a Random Forest as an independent reference.

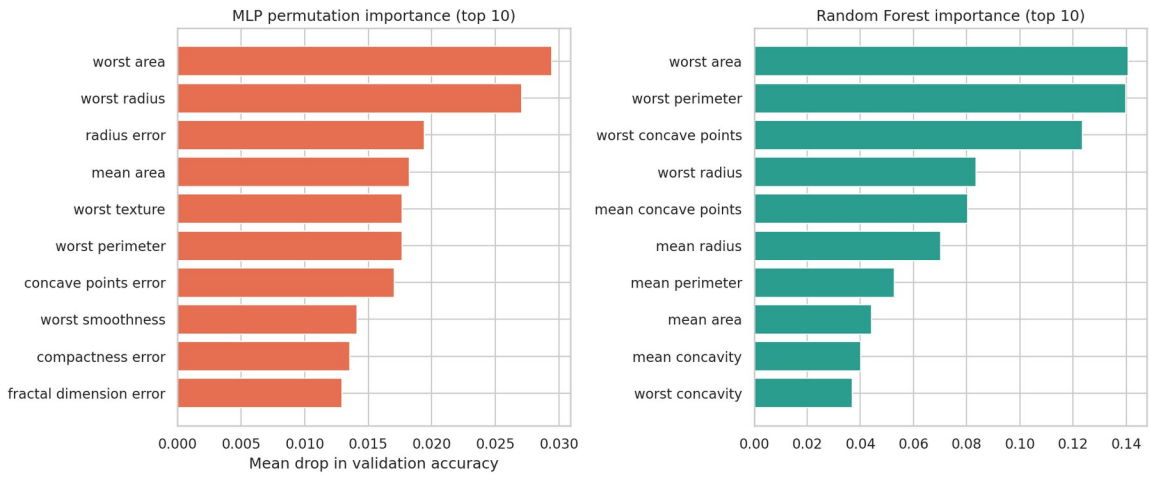


Figure 10: Feature importance - MLP permutation importance and Random Forest importance

Both methods place the size-related 'worst' features at the top (MLP: worst area, worst radius, radius error, mean area, worst texture; Random Forest: worst area, worst perimeter, worst concave points, worst radius, mean concave points), consistent with the mutual-information ranking in Figure 8. Permutation importance is small and spread out for the MLP because correlated features share the same information - shuffling one is compensated by its correlated partners.

6. Result Visualization and Interpretation

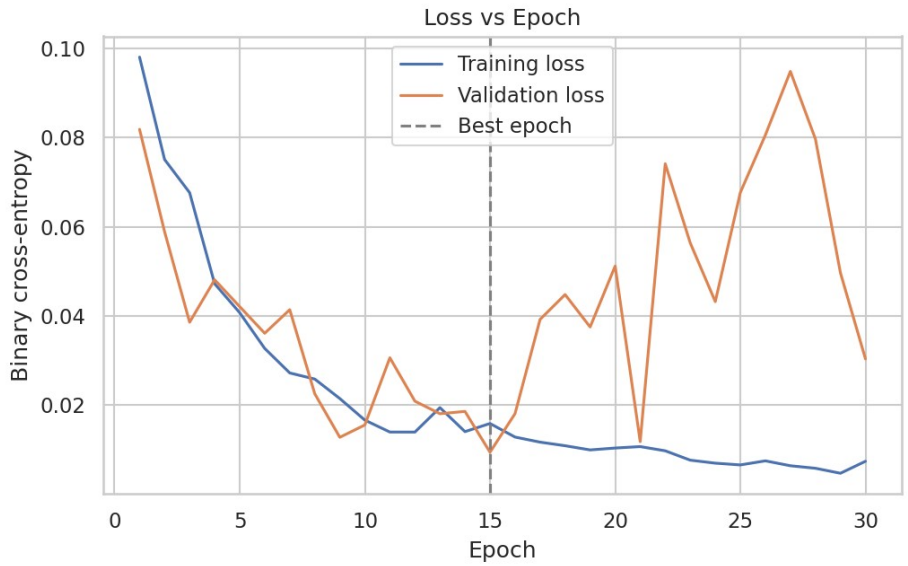


Figure 11: Loss vs Epoch

Training loss decreases steadily as the network learns. The validation loss reaches its minimum at epoch 15 and then stops improving, so early stopping (patience 15) halted training at epoch 30 and restored the weights from epoch 15. The validation curve is noisier

than the training curve because the validation set is small (85 samples) and the learning rate is 0.01; early stopping prevents the network from overfitting the training data.

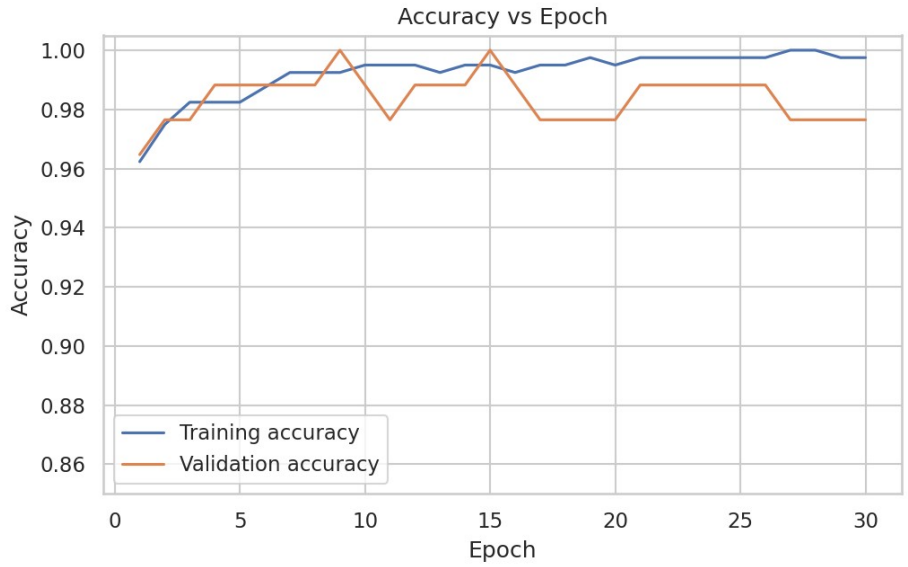


Figure 12: Accuracy vs Epoch

Accuracy at the best epoch is 99.5% (training) and 100.0% (validation). The model learns the task quickly because the classes are well separated (Figure 5).

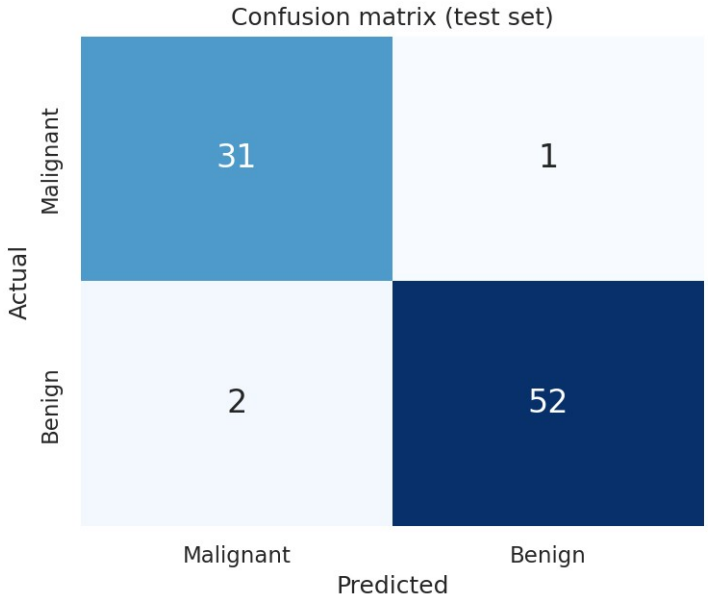


Figure 13: Confusion matrix on the test set

Of 86 test patients, 52 benign and 31 malignant cases were classified correctly; 1 malignant case(s) were wrongly predicted as benign (false positive for the Benign class) and 2 benign case(s) as malignant. In a clinical setting a malignant tumour predicted as benign is the most serious error, so this count should be minimized, for example by lowering the decision threshold.

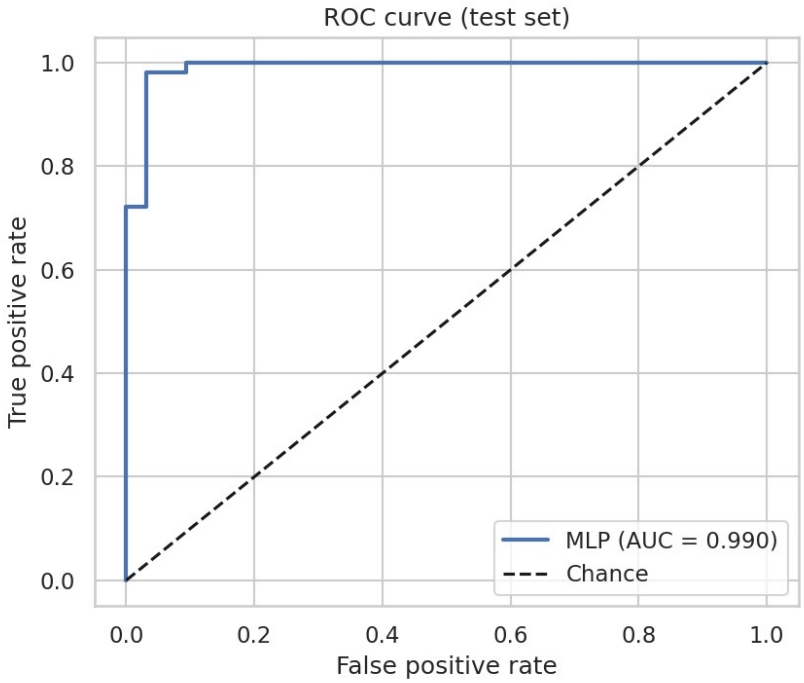


Figure 14: ROC curve and AUC on the test set

The ROC curve hugs the top-left corner and the area under the curve is 0.990, meaning the model ranks a random benign case above a random malignant case with probability 99.0%.

Table 7: Test-set performance summary

Metric	MLP (test)	Logistic regression baseline (test)
Accuracy	0.9651	0.9884
Precision (Benign)	0.9811	-
Recall (Benign)	0.9630	-
F1-score	0.9720	-
ROC-AUC	0.9902	0.9948

The logistic regression baseline performs comparably to (or slightly better than) the MLP. This is a common finding on small, well-separated tabular datasets and shows that a deep model is not automatically superior; its main benefit here is as a learning exercise and as a base for larger data. With only 86 test samples, a difference of one patient changes accuracy by about 1.2 percentage points.

7. Conclusion and Future Scope

7.1 Conclusion

- **What was learned:** The dataset is clean (no missing values or duplicates), moderately imbalanced, with skewed features and many outliers corresponding to large tumours.

- **Key patterns:** Size and shape features (radius, perimeter, area, concavity, concave points - especially the 'worst' values) differ strongly between malignant and benign tumours; these features are highly inter-correlated.
- **Model performance:** The tuned MLP reached 96.5% accuracy, 0.972 F1 and 0.990 AUC on unseen test data, comparable to a logistic regression baseline.
- **Challenges and limitations:** Small data size (569 rows) makes results sensitive to the particular split; heavy multicollinearity; the small validation set limits reliable tuning; and the model is not a medical-grade tool.

7.2 Future scope

- Use k-fold cross-validation or repeated splits to obtain confidence intervals instead of a single split.
- Remove redundant features with PCA or correlation filtering and compare model performance.
- Tune hyperparameters with Keras Tuner or Optuna, and compare with gradient boosting (XGBoost / LightGBM).
- Use explainability tools such as SHAP or LIME to explain individual predictions to clinicians.
- Calibrate probabilities and choose a decision threshold that prioritises recall for malignant cases.
- Train a CNN directly on the FNA images (a bigger, external dataset) and validate on data from other hospitals.

References

[1] W. H. Wolberg, W. N. Street, and O. L. Mangasarian, "Nuclear feature extraction for breast tumor diagnosis," in Proc. IS&T/SPIE Int. Symp. Electronic Imaging: Science and Technology, vol. 1905, 1993, pp. 861-870.

[2] D. Dua and C. Graff, "UCI Machine Learning Repository: Breast Cancer Wisconsin (Diagnostic) Data Set," Univ. of California, Irvine, 2019. [Online]. Available: https://archive.ics.uci.edu/ml

[3] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825-2830, 2011.

[4] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," arXiv:1412.6980, 2014.

[5] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," J. Mach. Learn. Res., vol. 15, pp. 1929-1958, 2014.

[6] B. Fry, Visualizing Data: Exploring and Explaining Data with the Processing Environment. Sebastopol, CA, USA: O'Reilly Media, 2007.

[7] M. Abadi et al., "TensorFlow: Large-scale machine learning on heterogeneous systems," 2015. [Online]. Available: https://www.tensorflow.org

Appendix - Source Code

The complete project, including dataset, notebooks/scripts, results and this report, is available at: https://github.com/<your-username>/breast-cancer-eda-mlp

src/config.py

```
"""Central configuration so every script uses the same paths and seeds."""
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
DATA_DIR = ROOT / "data"
PROC_DIR = DATA_DIR / "processed"
RESULTS_DIR = ROOT / "results"
FIG_DIR = RESULTS_DIR / "figures"
for _d in (DATA_DIR, PROC_DIR, RESULTS_DIR, FIG_DIR):
    _d.mkdir(parents=True, exist_ok=True)

SEED = 42
RAW_CSV = DATA_DIR / "breast_cancer_wisconsin.csv"
TARGET = "diagnosis"          # 0 = Malignant, 1 = Benign (UCI WDBC coding used by scikit-learn)
CLASS_NAMES = ["Malignant", "Benign"]
```

src/eda_visualization.py

```
"""
Steps 1-3: Data acquisition, EDA, preprocessing and visualisation.

Maps to the 7 stages of data visualisation (Ben Fry):
  1 Acquire  2 Parse  3 Filter  4 Mine  5 Represent  6 Refine  7 Interact

Run:  python src/eda_visualization.py
"""
import json
import warnings

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from scipy import stats
from sklearn.datasets import load_breast_cancer
from sklearn.feature_selection import mutual_info_classif
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

from config import CLASS_NAMES, FIG_DIR, PROC_DIR, RAW_CSV, RESULTS_DIR, SEED, TARGET

warnings.filterwarnings("ignore")
sns.set_theme(style="whitegrid", context="notebook")
PALETTE = {"Malignant": "#d62728", "Benign": "#1f77b4"}


def save(fig, name):
    fig.tight_layout()
    fig.savefig(FIG_DIR / name, dpi=150, bbox_inches="tight")
    plt.close(fig)


# ---------------------------------------------------------------- Stage 1: ACQUIRE
def acquire():
    """Load the UCI Breast Cancer Wisconsin (Diagnostic) dataset and cache it as CSV."""
    ds = load_breast_cancer(as_frame=True)
    df = ds.frame.rename(columns={"target": TARGET})
    df.to_csv(RAW_CSV, index=False)
    return df


# ---------------------------------------------------------------- Stage 2: PARSE
def parse(df):
    """Structure, types, missing values, duplicates, summary statistics."""
    info = {
        "rows": int(df.shape[0]),
        "columns": int(df.shape[1]),
```

```
        "n_features": int(df.shape[1] - 1),
        "missing_total": int(df.isna().sum().sum()),
        "duplicates": int(df.duplicated().sum()),
        "class_counts": {CLASS_NAMES[k]: int(v) for k, v in
df[TARGET].value_counts().sort_index().items()},
    }
    df.describe().T.round(3).to_csv(RESULTS_DIR / "summary_statistics.csv")
    return info


# ---------------------------------------------------------------- Stage 3: FILTER
def iqr_outliers(X):
    q1, q3 = X.quantile(.25), X.quantile(.75)
    iqr = q3 - q1
    return ((X < q1 - 1.5 * iqr) | (X > q3 + 1.5 * iqr)).sum()


def filter_data(df):
    """Remove duplicates; report outliers (kept - they are clinically valid)."""
    df = df.drop_duplicates().reset_index(drop=True)
    out = iqr_outliers(df.drop(columns=TARGET))
    return df, out


# ---------------------------------------------------------------- Stage 4: MINE
def mine(df):
    """Statistics: skewness, normality, correlation, mutual information, class t-tests."""
    X, y = df.drop(columns=TARGET), df[TARGET]
    skew = X.skew().sort_values(ascending=False)
    shapiro = {c: stats.shapiro(X[c])[1] for c in X.columns}
    mi = pd.Series(mutual_info_classif(X, y, random_state=SEED),
index=X.columns).sort_values(ascending=False)
    ttest = {c: stats.ttest_ind(X.loc[y == 0, c], X.loc[y == 1, c], equal_var=False)[1] for c in
X.columns}
    corr = X.corr()
    target_corr = X.apply(lambda s: s.corr(y)).sort_values()
    pairs = corr.where(np.triu(np.ones(corr.shape),
1).astype(bool)).stack().sort_values(ascending=False)
    return {"skew": skew, "shapiro": shapiro, "mi": mi, "ttest": ttest, "corr": corr,
            "target_corr": target_corr, "top_pairs": pairs.head(5)}


# ---------------------------------------------------------------- Stage 5: REPRESENT
def represent(df, m):
    X, y = df.drop(columns=TARGET), df[TARGET]
    labels = y.map(dict(enumerate(CLASS_NAMES)))
    mean_cols = [c for c in X.columns if c.startswith("mean ")]

    # V1 - Class distribution (discrete: Bernoulli / Binomial)
    fig, ax = plt.subplots(1, 2, figsize=(9, 4))
    counts = labels.value_counts()[CLASS_NAMES]
    sns.barplot(x=counts.index, y=counts.values, palette=PALETTE, ax=ax[0])
    for i, v in enumerate(counts.values):
        ax[0].text(i, v + 4, str(v), ha="center", fontweight="bold")
    ax[0].set(title="Class distribution (count)", xlabel="Diagnosis", ylabel="Number of patients")
    ax[1].pie(counts.values, labels=counts.index, autopct="%1.1f%%", colors=[PALETTE[c] for c in
counts.index], startangle=90)
    ax[1].set_title("Class proportion")
    save(fig, "v1_class_distribution.png")

    # V2 - Histograms + KDE + fitted normal curve (continuous distributions)
    sel = ["mean radius", "mean texture", "mean area", "mean concavity"]
    fig, axes = plt.subplots(2, 2, figsize=(11, 8))
    for ax, c in zip(axes.ravel(), sel):
        sns.histplot(X[c], kde=True, stat="density", bins=30, color="#4c72b0", ax=ax)
        xs = np.linspace(X[c].min(), X[c].max(), 200)
        ax.plot(xs, stats.norm.pdf(xs, X[c].mean(), X[c].std()), "r--", label="Normal fit")
        ax.set_title(f"{c}  (skew={X[c].skew():.2f})")
        ax.legend()
    fig.suptitle("Univariate distributions with fitted normal curve", y=1.02, fontsize=14)
    save(fig, "v2_histograms_normal_fit.png")

    # V3 - Boxplots by class (outliers + class separation)
    fig, axes = plt.subplots(2, 3, figsize=(13, 7))
    for ax, c in zip(axes.ravel(), ["mean radius", "mean texture", "mean perimeter", "mean area",
"mean concavity", "mean concave points"]):
        sns.boxplot(x=labels, y=X[c], order=CLASS_NAMES, palette=PALETTE, ax=ax)
        ax.set(title=c, xlabel="", ylabel="")
    fig.suptitle("Boxplots by diagnosis", y=1.02, fontsize=14)
    save(fig, "v3_boxplots_by_class.png")

    # V4 - Correlation heatmap (mean features)
```

```
    fig, ax = plt.subplots(figsize=(9, 7.5))
    sns.heatmap(X[mean_cols].corr(), annot=True, fmt=".2f", cmap="coolwarm", center=0,
annot_kws={"size": 7}, ax=ax)
    ax.set_title("Correlation heatmap - 'mean' features")
    save(fig, "v4_correlation_heatmap.png")

    # V5 - Scatterplot radius vs concave points coloured by class
    fig, ax = plt.subplots(figsize=(7, 5.5))
    sns.scatterplot(data=X.assign(Diagnosis=labels), x="mean radius", y="mean concave points",
hue="Diagnosis",
                    palette=PALETTE, alpha=.75, edgecolor="w", ax=ax)
    ax.set_title("Mean radius vs mean concave points")
    save(fig, "v5_scatter_radius_concavepoints.png")

    # V6 - Pairplot of 5 strongest features
    top5 = m["mi"].index[:5].tolist()
    g = sns.pairplot(X[top5].assign(Diagnosis=labels), hue="Diagnosis", palette=PALETTE,
corner=True, plot_kws={"s": 18, "alpha": .7})
    g.figure.suptitle("Pairplot of the 5 most informative features", y=1.02)
    g.figure.savefig(FIG_DIR / "v6_pairplot_top5.png", dpi=130, bbox_inches="tight")
    plt.close(g.figure)

    # V7 - Q-Q plots (normality check for continuous distributions)
    fig, axes = plt.subplots(1, 3, figsize=(13, 4))
    for ax, c in zip(axes, ["mean radius", "mean area", "mean smoothness"]):
        stats.probplot(X[c], dist="norm", plot=ax)
        ax.set_title(f"Q-Q plot: {c}")
    save(fig, "v7_qq_plots.png")

    # V8 - Mutual information / feature-target correlation (Mine -> Represent)
    fig, ax = plt.subplots(figsize=(8, 8))
    mi = m["mi"].sort_values()
    ax.barh(mi.index, mi.values, color="#2a9d8f")
    ax.set(title="Mutual information with diagnosis", xlabel="Mutual information")
    save(fig, "v8_mutual_information.png")


# ---------------------------------------------------------------- Stage 6: REFINE
def refine(df):
    """Show the effect of preprocessing: before / after scaling + outlier clipping."""
    X = df.drop(columns=TARGET)
    cols = ["mean radius", "mean area", "mean smoothness", "mean concavity"]
    fig, ax = plt.subplots(1, 2, figsize=(12, 4.5))
    sns.boxplot(data=X[cols], ax=ax[0]); ax[0].set_title("Before scaling (different units)")
    Xs = pd.DataFrame(StandardScaler().fit_transform(X), columns=X.columns)[cols]
    sns.boxplot(data=Xs, ax=ax[1]); ax[1].set_title("After standardisation (z-score)")
    save(fig, "v9_before_after_scaling.png")


# ---------------------------------------------------------------- Stage 7: INTERACT
def interact(df):
    """Interactive Plotly scatter (hover, zoom, legend toggle). Optional dependency."""
    try:
        import plotly.express as px
    except ImportError:
        print("plotly not installed -> skipping interactive chart (pip install plotly)")
        return False
    d = df.assign(Diagnosis=df[TARGET].map(dict(enumerate(CLASS_NAMES))))
    fig = px.scatter(d, x="mean radius", y="mean concave points", color="Diagnosis", size="mean
area",
                     hover_data=["mean texture", "mean perimeter"],
                     color_discrete_map=PALETTE, title="Interactive: radius vs concave points
(bubble = area)")
    fig.write_html(RESULTS_DIR / "interactive_scatter.html")
    return True


# ---------------------------------------------------------------- Preprocess + split
def preprocess_split(df):
    """Clip extreme outliers (3*IQR, fences from TRAIN only), standardise (fit on TRAIN only),
70/15/15 split."""
    X, y = df.drop(columns=TARGET), df[TARGET]
    X_tr, X_tmp, y_tr, y_tmp = train_test_split(X, y, test_size=.30, stratify=y, random_state=SEED)
    X_val, X_te, y_val, y_te = train_test_split(X_tmp, y_tmp, test_size=.50, stratify=y_tmp,
random_state=SEED)

    q1, q3 = X_tr.quantile(.25), X_tr.quantile(.75)
    lo, hi = q1 - 3 * (q3 - q1), q3 + 3 * (q3 - q1)
    n_clipped = int(((X_tr < lo) | (X_tr > hi)).sum().sum())
    X_tr, X_val, X_te = (s.clip(lo, hi, axis=1) for s in (X_tr, X_val, X_te))

    sc = StandardScaler().fit(X_tr)
```

```
    out = {}
    for name, Xs, ys in (("train", X_tr, y_tr), ("val", X_val, y_val), ("test", X_te, y_te)):
        arr = pd.DataFrame(sc.transform(Xs), columns=X.columns, index=Xs.index).assign(**{TARGET:
ys.values})
        arr.to_csv(PROC_DIR / f"{name}.csv", index=False)
        out[name] = int(len(arr))
    out["extreme_values_clipped"] = n_clipped
    return out


def main():
    df = acquire()                                      # 1 Acquire
    info = parse(df)                                    # 2 Parse
    df, out = filter_data(df)                           # 3 Filter
    m = mine(df)                                        # 4 Mine
    represent(df, m)                                    # 5 Represent
    refine(df)                                          # 6 Refine
    info["interactive_html"] = interact(df)             # 7 Interact
    info["split_sizes"] = preprocess_split(df)
    info["outliers_iqr_total"] = int(out.sum())
    info["outliers_top5"] = out.sort_values(ascending=False).head(5).astype(int).to_dict()
    info["skew_top3"] = m["skew"].head(3).round(2).to_dict()
    info["n_nonnormal_shapiro"] = int(sum(p < .05 for p in m["shapiro"].values()))
    info["n_sig_ttest"] = int(sum(p < .05 for p in m["ttest"].values()))
    info["mi_top5"] = m["mi"].head(5).round(3).to_dict()
    info["target_corr_top"] = {"most_negative": m["target_corr"].head(3).round(3).to_dict(),
                               "most_positive": m["target_corr"].tail(3).round(3).to_dict()}
    info["top_corr_pairs"] = {" | ".join(k): round(float(v), 3) for k, v in m["top_pairs"].items()}
    (RESULTS_DIR / "eda_info.json").write_text(json.dumps(info, indent=2))
    print(json.dumps(info, indent=2))


if __name__ == "__main__":
    main()
```

src/numpy_mlp.py

```
"""
Minimal NumPy MLP (ReLU hidden layers, sigmoid output, dropout, Adam, early stopping).

It mirrors the Keras model in `model_mlp.py` and is used ONLY as a fallback when
TensorFlow is not installed, so the whole pipeline still runs end-to-end.
"""
import numpy as np


class NumpyMLP:
    def __init__(self, n_in, hidden=(32, 16), dropout=0.2, lr=1e-3, seed=42):
        self.rng = np.random.default_rng(seed)
        sizes = [n_in, *hidden, 1]
        # He initialisation
        self.W = [self.rng.normal(0, np.sqrt(2 / sizes[i]), (sizes[i], sizes[i + 1])) for i in
range(len(sizes) - 1)]
        self.b = [np.zeros(sizes[i + 1]) for i in range(len(sizes) - 1)]
        self.dropout, self.lr = dropout, lr
        self.m = [np.zeros_like(p) for p in self.W + self.b]
        self.v = [np.zeros_like(p) for p in self.W + self.b]
        self.t = 0

    @staticmethod
    def _sigmoid(z):
        return 1 / (1 + np.exp(-np.clip(z, -50, 50)))

    def _forward(self, X, train):
        acts, masks = [X], []
        for i in range(len(self.W) - 1):
            a = np.maximum(0, acts[-1] @ self.W[i] + self.b[i])
            if train and self.dropout > 0:
                mask = (self.rng.random(a.shape) > self.dropout) / (1 - self.dropout)
                a = a * mask
                masks.append(mask)
            else:
                masks.append(None)
            acts.append(a)
        out = self._sigmoid(acts[-1] @ self.W[-1] + self.b[-1]).ravel()
        return out, acts, masks

    def predict_proba(self, X):
        return self._forward(X, train=False)[0]
```

```
    @staticmethod
    def _bce(y, p):
        p = np.clip(p, 1e-7, 1 - 1e-7)
        return float(-np.mean(y * np.log(p) + (1 - y) * np.log(1 - p)))

    def _step(self, Xb, yb):
        p, acts, masks = self._forward(Xb, train=True)
        delta = ((p - yb) / len(yb)).reshape(-1, 1)
        gW, gb = [None] * len(self.W), [None] * len(self.b)
        for i in reversed(range(len(self.W))):
            gW[i] = acts[i].T @ delta
            gb[i] = delta.sum(0)
            if i > 0:
                delta = (delta @ self.W[i].T) * (acts[i] > 0)
                if masks[i - 1] is not None:
                    delta = delta * masks[i - 1]
        self.t += 1
        for k, (p_, g) in enumerate(zip(self.W + self.b, gW + gb)):   # Adam update (in place)
            self.m[k] = 0.9 * self.m[k] + 0.1 * g
            self.v[k] = 0.999 * self.v[k] + 0.001 * g * g
            mh = self.m[k] / (1 - 0.9 ** self.t)
            vh = self.v[k] / (1 - 0.999 ** self.t)
            p_ -= self.lr * mh / (np.sqrt(vh) + 1e-7)

    def fit(self, Xtr, ytr, Xval, yval, epochs=100, batch_size=32, patience=15):
        hist = {"loss": [], "val_loss": [], "accuracy": [], "val_accuracy": []}
        best, best_w, wait = np.inf, None, 0
        for _ in range(epochs):
            idx = self.rng.permutation(len(ytr))
            for s in range(0, len(idx), batch_size):
                b = idx[s:s + batch_size]
                self._step(Xtr[b], ytr[b])
            ptr, pval = self.predict_proba(Xtr), self.predict_proba(Xval)
            hist["loss"].append(self._bce(ytr, ptr)); hist["val_loss"].append(self._bce(yval, pval))
            hist["accuracy"].append(float(np.mean((ptr > .5) == ytr)))
            hist["val_accuracy"].append(float(np.mean((pval > .5) == yval)))
            if hist["val_loss"][-1] < best - 1e-5:
                best, wait = hist["val_loss"][-1], 0
                best_w = ([w.copy() for w in self.W], [b_.copy() for b_ in self.b])
            else:
                wait += 1
                if wait >= patience:
                    break
        if best_w:
            self.W, self.b = best_w
        return hist
```

src/model_mlp.py

```
"""
Steps 4-5: MLP model (Keras/TensorFlow), basic hyper-parameter tuning,
feature importance and evaluation visualisations.

Run:  python src/model_mlp.py          (after src/eda_visualization.py)

If TensorFlow is not installed, an equivalent NumPy MLP (numpy_mlp.py) is used and
the backend name is stored in results/metrics.json.
"""
import itertools
import json
import random
import warnings

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (accuracy_score, auc, classification_report, confusion_matrix, f1_score,
                             precision_score, recall_score, roc_curve)

from config import CLASS_NAMES, FIG_DIR, PROC_DIR, RESULTS_DIR, SEED, TARGET
from numpy_mlp import NumpyMLP

warnings.filterwarnings("ignore")
sns.set_theme(style="whitegrid")
np.random.seed(SEED); random.seed(SEED)
```

```
try:                                    # ---- backend selection
    import tensorflow as tf
    from tensorflow import keras
    tf.random.set_seed(SEED)
    BACKEND = f"Keras/TensorFlow {tf.__version__}"
except ImportError:
    BACKEND = "NumPy MLP (TensorFlow not installed)"

EPOCHS, BATCH, PATIENCE = 100, 32, 15


def load_data():
    tr, va, te = (pd.read_csv(PROC_DIR / f"{n}.csv") for n in ("train", "val", "test"))
    feats = [c for c in tr.columns if c != TARGET]
    return (tr[feats].values, tr[TARGET].values.astype(float), va[feats].values,
va[TARGET].values.astype(float),
            te[feats].values, te[TARGET].values.astype(float), feats)


class Model:
    """Thin wrapper giving Keras and NumPy models the same interface."""

    def __init__(self, n_in, hidden, dropout, lr):
        self.hidden, self.dropout, self.lr = hidden, dropout, lr
        if BACKEND.startswith("Keras"):
            keras.backend.clear_session()
            layers = [keras.layers.Input((n_in,))]
            for h in hidden:                        # Dense -> ReLU -> Dropout blocks
                layers += [keras.layers.Dense(h, activation="relu"), keras.layers.Dropout(dropout)]
            layers.append(keras.layers.Dense(1, activation="sigmoid"))
            self.m = keras.Sequential(layers)
            self.m.compile(optimizer=keras.optimizers.Adam(lr), loss="binary_crossentropy",
metrics=["accuracy"])
        else:
            self.m = NumpyMLP(n_in, hidden, dropout, lr, SEED)

    def fit(self, Xtr, ytr, Xva, yva):
        if BACKEND.startswith("Keras"):
            es = keras.callbacks.EarlyStopping(monitor="val_loss", patience=PATIENCE,
restore_best_weights=True)
            h = self.m.fit(Xtr, ytr, validation_data=(Xva, yva), epochs=EPOCHS, batch_size=BATCH,
callbacks=[es], verbose=0)
            return {k: [float(x) for x in v] for k, v in h.history.items()}
        return self.m.fit(Xtr, ytr, Xva, yva, EPOCHS, BATCH, PATIENCE)

    def predict_proba(self, X):
        if BACKEND.startswith("Keras"):
            return self.m.predict(X, verbose=0).ravel()
        return self.m.predict_proba(X)

    def summary_text(self):
        if BACKEND.startswith("Keras"):
            lines = []
            self.m.summary(print_fn=lines.append)
            return "\n".join(lines)
        return f"NumPy MLP: {self.hidden}, dropout={self.dropout}, lr={self.lr}"


def tune(Xtr, ytr, Xva, yva):
    """Basic grid search on the VALIDATION set (test set is never touched)."""
    grid = {"hidden": [(16, 8), (32, 16), (64, 32)], "dropout": [0.2, 0.3], "lr": [1e-3, 1e-2]}
    rows = []
    for hidden, dr, lr in itertools.product(grid["hidden"], grid["dropout"], grid["lr"]):
        mdl = Model(Xtr.shape[1], hidden, dr, lr)
        hist = mdl.fit(Xtr, ytr, Xva, yva)
        rows.append({"hidden_units": str(hidden), "dropout": dr, "learning_rate": lr,
                     "best_val_loss": round(min(hist["val_loss"]), 4),
                     "best_val_acc": round(max(hist["val_accuracy"] if "val_accuracy" in hist else
hist["val_acc"]), 4),
                     "epochs_run": len(hist["loss"])})
        print(rows[-1])
    res = pd.DataFrame(rows).sort_values("best_val_loss").reset_index(drop=True)
    res.to_csv(RESULTS_DIR / "hyperparameter_tuning.csv", index=False)
    best = res.iloc[0]
    return eval(best["hidden_units"]), float(best["dropout"]), float(best["learning_rate"]), res


def permutation_importance(model, X, y, feats, n_rep=20):
    """Drop in accuracy when a feature is shuffled (model-agnostic feature importance)."""
    rng = np.random.default_rng(SEED)
    base = accuracy_score(y, model.predict_proba(X) > .5)
```

```
    imp = []
    for j in range(X.shape[1]):
        drops = []
        for _ in range(n_rep):
            Xp = X.copy(); Xp[:, j] = rng.permutation(Xp[:, j])
            drops.append(base - accuracy_score(y, model.predict_proba(Xp) > .5))
        imp.append(np.mean(drops))
    return pd.Series(imp, index=feats).sort_values(ascending=False)


def plots(hist, y_te, p_te, imp, rf_imp):
    ep = np.arange(1, len(hist["loss"]) + 1)
    acc_k, vacc_k = ("accuracy", "val_accuracy") if "val_accuracy" in hist else ("acc", "val_acc")

    fig, ax = plt.subplots(figsize=(7, 4.5))                                    # R1 loss
    ax.plot(ep, hist["loss"], label="Training loss"); ax.plot(ep, hist["val_loss"],
label="Validation loss")
    ax.axvline(int(np.argmin(hist["val_loss"])) + 1, color="grey", ls="--", label="Best epoch")
    ax.set(title="Loss vs Epoch", xlabel="Epoch", ylabel="Binary cross-entropy"); ax.legend()
    fig.tight_layout(); fig.savefig(FIG_DIR / "r1_loss_vs_epoch.png", dpi=150); plt.close(fig)

    fig, ax = plt.subplots(figsize=(7, 4.5))                                    # R2 accuracy
    ax.plot(ep, hist[acc_k], label="Training accuracy"); ax.plot(ep, hist[vacc_k], label="Validation
accuracy")
    ax.set(title="Accuracy vs Epoch", xlabel="Epoch", ylabel="Accuracy", ylim=(.85, 1.005));
ax.legend()
    fig.tight_layout(); fig.savefig(FIG_DIR / "r2_accuracy_vs_epoch.png", dpi=150); plt.close(fig)

    cm = confusion_matrix(y_te, p_te > .5)                                      # R3 confusion matrix
    fig, ax = plt.subplots(figsize=(5.2, 4.5))
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", xticklabels=CLASS_NAMES,
yticklabels=CLASS_NAMES, cbar=False,
                annot_kws={"size": 16}, ax=ax)
    ax.set(title="Confusion matrix (test set)", xlabel="Predicted", ylabel="Actual")
    fig.tight_layout(); fig.savefig(FIG_DIR / "r3_confusion_matrix.png", dpi=150); plt.close(fig)

    fpr, tpr, _ = roc_curve(y_te, p_te); roc_auc = auc(fpr, tpr)                # R4 ROC
    fig, ax = plt.subplots(figsize=(5.8, 5))
    ax.plot(fpr, tpr, lw=2, label=f"MLP (AUC = {roc_auc:.3f})"); ax.plot([0, 1], [0, 1], "k--",
label="Chance")
    ax.set(title="ROC curve (test set)", xlabel="False positive rate", ylabel="True positive rate");
ax.legend(loc="lower right")
    fig.tight_layout(); fig.savefig(FIG_DIR / "r4_roc_curve.png", dpi=150); plt.close(fig)

    fig, ax = plt.subplots(1, 2, figsize=(13, 5.5))                             # R5 feature importance
    t = imp.head(10)[::-1]; ax[0].barh(t.index, t.values, color="#e76f51"); ax[0].set_title("MLP
permutation importance (top 10)")
    ax[0].set_xlabel("Mean drop in validation accuracy")
    t = rf_imp.head(10)[::-1]; ax[1].barh(t.index, t.values, color="#2a9d8f");
ax[1].set_title("Random Forest importance (top 10)")
    fig.tight_layout(); fig.savefig(FIG_DIR / "r5_feature_importance.png", dpi=150); plt.close(fig)
    return float(roc_auc)


def main():
    Xtr, ytr, Xva, yva, Xte, yte, feats = load_data()
    print("Backend:", BACKEND)

    hidden, dropout, lr, tuning = tune(Xtr, ytr, Xva, yva)                      # hyper-parameter
tuning
    model = Model(Xtr.shape[1], hidden, dropout, lr)                            # final model
    hist = model.fit(Xtr, ytr, Xva, yva)

    p_te = model.predict_proba(Xte); pred = (p_te > .5).astype(int)
    imp = permutation_importance(model, Xva, yva, feats)
    rf = RandomForestClassifier(300, random_state=SEED).fit(Xtr, ytr)
    rf_imp = pd.Series(rf.feature_importances_, index=feats).sort_values(ascending=False)
    roc_auc = plots(hist, yte, p_te, imp, rf_imp)

    lr_model = LogisticRegression(max_iter=1000).fit(Xtr, ytr)                  # baseline for
comparison
    base_p = lr_model.predict_proba(Xte)[:, 1]
    tn, fp, fn, tp = confusion_matrix(yte, pred).ravel()
    metrics = {
        "backend": BACKEND,
        "architecture": {"input": len(feats), "hidden": list(hidden), "dropout": dropout,
"learning_rate": lr,
                         "output": "1 sigmoid unit", "optimizer": "Adam", "loss": "binary cross-
entropy",
                         "batch_size": BATCH, "max_epochs": EPOCHS, "early_stopping_patience":
PATIENCE},
        "epochs_run": len(hist["loss"]),
```

```
        "best_epoch": int(np.argmin(hist["val_loss"])) + 1,
        "test": {"accuracy": accuracy_score(yte, pred), "precision": precision_score(yte, pred),
                 "recall": recall_score(yte, pred), "f1": f1_score(yte, pred), "auc": roc_auc,
                 "tn": int(tn), "fp": int(fp), "fn": int(fn), "tp": int(tp)},
        "baseline_logreg_test": {"accuracy": accuracy_score(yte, base_p > .5), "auc":
float(auc(*roc_curve(yte, base_p)[:2]))},
        # accuracies at the best epoch (the weights that were restored by early stopping)
        "final_train_acc": hist.get("accuracy", hist.get("acc"))[int(np.argmin(hist["val_loss"]))],
        "final_val_acc": hist.get("val_accuracy", hist.get("val_acc"))
[int(np.argmin(hist["val_loss"]))],
        "top_features_mlp": imp.head(5).round(4).to_dict(),
        "top_features_rf": rf_imp.head(5).round(4).to_dict(),
        "tuning_top3": tuning.head(3).to_dict(orient="records"),
        "tuning_n_configs": int(len(tuning)),
        "classification_report": classification_report(yte, pred, target_names=CLASS_NAMES,
output_dict=True),
        "model_summary": model.summary_text(),
    }
    (RESULTS_DIR / "metrics.json").write_text(json.dumps(metrics, indent=2, default=float))
    print(json.dumps({k: v for k, v in metrics.items() if k not in ("model_summary",
"classification_report")}, indent=2, default=float))


if __name__ == "__main__":
    main()
```